NYC_Taxi_Pricing_Model

May 12, 2025

1 Proyecto Automatidata

1.1 Curso 5 - Análisis de Regresión: Simplificar relaciones complejas de datos

1.1.1 Autor: Victor Hugo Villegas

1.1.2 Fecha: 02 de mayo de 2025

1.1.3 Introducción

La firma de consultoría de datos **Automatidata** te ha contratado como el nuevo miembro de su equipo de análisis de datos.
Su cliente más reciente, la **Comisión de Taxis y Limusinas de Nueva York (NYC TLC)**, quiere que el equipo de Automatidata construya un **modelo de regresión lineal múltiple** para **predecir las tarifas de taxi**, utilizando datos recopilados durante un año.

El equipo está avanzando en el proyecto y ha completado:
Plan de acción inicial
Codificación inicial en Python
Análisis exploratorio de datos (EDA)
Prueba A/B

1.1.4 Siguiente paso: Predicción de tarifas de taxi

El equipo de Automatidata ha revisado los resultados de la prueba A/B.
Ahora es el momento de **trabajar en la predicción de los montos de tarifas de taxi**.

Has impresionado a tus colegas con tu dedicación y atención al detalle.
El equipo de datos confía en que estás listo para **construir el modelo de regresión** y actualizar al cliente **NYC TLC** sobre los avances del proyecto.

1.1.5 Tarea a realizar

Un **cuaderno de Jupyter Notebook** ha sido estructurado y preparado para ayudarte en este proyecto.
Completa las siguientes preguntas dentro del notebook y continúa con el desarrollo del modelo.

¡Adelante con el análisis!

1.2 Proyecto de fin de curso 5: Construcción de un modelo de regresión lineal múltiple

1.2.1 Introducción

En esta actividad, construirás un **modelo de regresión lineal múltiple**. Como has aprendido, este tipo de regresión ayuda a estimar la relación lineal entre una **variable dependiente continua** y **dos o más variables independientes**.

Para los profesionales de datos, esta técnica es muy útil porque permite considerar múltiples variables y realizar análisis más **completos y flexibles**.

Objetivo del proyecto:
Practicar la planificación y construcción de un modelo de regresión lineal múltiple según una **necesidad comercial específica**.
Simular un **proceso de trabajo real** como los que enfrentarás en tu carrera como profesional de datos.
Demostrar conocimientos en **EDA (Exploratory Data Analysis)** y modelado de regresión lineal múltiple.

1.2.2 Estructura de la actividad

La actividad consta de **tres partes clave**:

Parte 1: EDA y Verificación de Supuestos del Modelo **¿Cuáles son algunos propósitos del EDA antes de construir un modelo de regresión lineal múltiple?**

Parte 2: Construcción y Evaluación del Modelo **¿Qué recursos utilizaste durante esta etapa?**

Parte 3: Interpretación de los Resultados del Modelo **¿Qué ideas clave surgieron de tu modelo?**
¿Qué recomendaciones comerciales propones basándote en los modelos construidos?

¡Continúa con el desarrollo del proyecto y optimiza tu modelo de regresión!

2 1. Cargar el dataset limpio y verificar estructura

2.0.1 Propósito de este paso:

Asegurar que no haya valores nulos para evitar errores en el análisis.
Revisar los tipos de datos y confirmar que están correctamente definidos.
Visualizar algunas filas para comprobar que la limpieza de datos se realizó correctamente.

Este paso es crucial para garantizar que el dataset esté listo para el análisis y modelado.

[3]:
```
# Importar librerías principales
import pandas as pd
import numpy as np
```

```python
# Cargar el dataset limpio
ruta = 'C:/Users/victo/OneDrive/Data_Science/Python/taxi_trip_data_clean.csv'
df = pd.read_csv(ruta)

# Verificamos las dimensiones del dataframe
print(f"Número de filas y columnas: {df.shape}")

# Mostramos las primeras filas
df.head()
```

```
Número de filas y columnas: (19143, 5)
```

[3]:

	trip_distance	trip_duration	hora_del_día	es_fin_de_semana	total_amount
0	3.34	14.066667	8	1	16.56
1	1.80	26.500000	14	0	20.80
2	1.00	7.200000	7	0	8.75
3	4.37	16.716667	23	1	17.80
4	2.30	8.000000	20	1	12.36

3 2. Información general del DataFrame

3.0.1 Este análisis te permitirá:

Examinar los tipos de datos por columna y asegurarte de que están correctamente definidos.
Revisar la cantidad de valores no nulos para detectar posibles datos faltantes.
Identificar errores de tipo de datos, como fechas almacenadas como `object` en lugar de `datetime`.

Este paso es clave para garantizar que el dataset esté limpio y listo para el análisis y modelado.

[4]:
```python
# Información general del DataFrame
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 19143 entries, 0 to 19142
Data columns (total 5 columns):
 #   Column            Non-Null Count  Dtype
---  ------            --------------  -----
 0   trip_distance     19143 non-null  float64
 1   trip_duration     19143 non-null  float64
 2   hora_del_día      19143 non-null  int64
 3   es_fin_de_semana  19143 non-null  int64
 4   total_amount      19143 non-null  float64
dtypes: float64(3), int64(2)
memory usage: 747.9 KB
```

4 3. Verificar valores nulos

4.0.1 Interpretación esperada:

Si todo está correcto, no deberían aparecer valores nulos en el dataset.
Las variables clave para el modelo, como `trip_distance` y `trip_duration`, deben estar en **formato numérico**.
Variables como `hora_del_día` o `es_fin_de_semana` pueden estar en formato **int, float o category**, según cómo fueron creadas.

Este paso es esencial para garantizar la integridad de los datos antes de proceder con el modelado.

[3]:
```
# Verificar si existen valores nulos
df.isnull().sum()
```

[3]:
```
trip_distance       0
trip_duration       0
hora_del_día        0
es_fin_de_semana    0
total_amount        0
dtype: int64
```

5 2. Selección de variables predictoras y variable objetivo

5.0.1 Objetivo del modelo

Queremos predecir la variable **`total_amount`** (monto total pagado por el viaje en taxi) a partir de algunas **variables independientes**.

Seleccionar las variables adecuadas mejorará la precisión del modelo de regresión lineal múltiple.

6 Variables seleccionadas

Tipo	Variable	Descripción breve
Predictora	`trip_distance`	Distancia recorrida en millas
Predictora	`trip_duration`	Duración del viaje en minutos (o segundos)
Predictora	`hora_del_día`	Hora del día del inicio del viaje (0-23)
Predictora	`es_fin_de_semana`	0 entre semana, 1 fin de semana
Objetivo	`total_amount`	Valor total pagado por el pasajero (tarifa final)

Estas variables serán utilizadas en el modelo de regresión para predecir el monto total del viaje en taxi.

7 Justificación técnica de las variables predictoras

7.0.1 Motivación de la selección

Las siguientes variables fueron elegidas por su influencia directa en el costo del viaje y su adecuada interpretación en un modelo de **regresión lineal múltiple**.

7.0.2 Variables predictoras y su impacto en la tarifa

- `trip_distance` → Altamente correlacionada con el costo del viaje. **Más distancia = mayor tarifa**.
- `trip_duration` → Complementa a la distancia. **En atascos, un viaje corto puede durar mucho y aumentar el total**.
- `hora_del_día` → Puede capturar **diferencias por tráfico o tarifas nocturnas**.
- `es_fin_de_semana` → Puede captar **variaciones tarifarias o de comportamiento de viaje**.

7.0.3 Validez técnica

Son **variables relevantes** con una interpretación lógica.
Están en **formatos adecuados** para la regresión lineal.
Contribuyen a mejorar la **precisión del modelo** en la predicción del `total_amount`.

Estas características aseguran un modelo sólido y con buena capacidad predictiva.

8 Definimos X (predictoras) y y (objetivo)

8.0.1 Variables en el modelo

`X` (Variables predictoras):
- `trip_distance` → Distancia recorrida en el viaje.
- `trip_duration` → Duración total del viaje.
- `hora_del_día` → Hora en la que comenzó el viaje.
- `es_fin_de_semana` → Indica si el viaje ocurrió en fin de semana.

`y` (Variable objetivo):
- `total_amount` → Monto total pagado por el pasajero.

Estas variables serán utilizadas en la regresión lineal múltiple para predecir la tarifa del viaje en taxi.

[5]:
```
# Separar variables predictoras (X) y variable objetivo (y)
X = df[['trip_distance', 'trip_duration', 'hora_del_día', 'es_fin_de_semana']]
y = df['total_amount']
```

9 3. Análisis de correlación

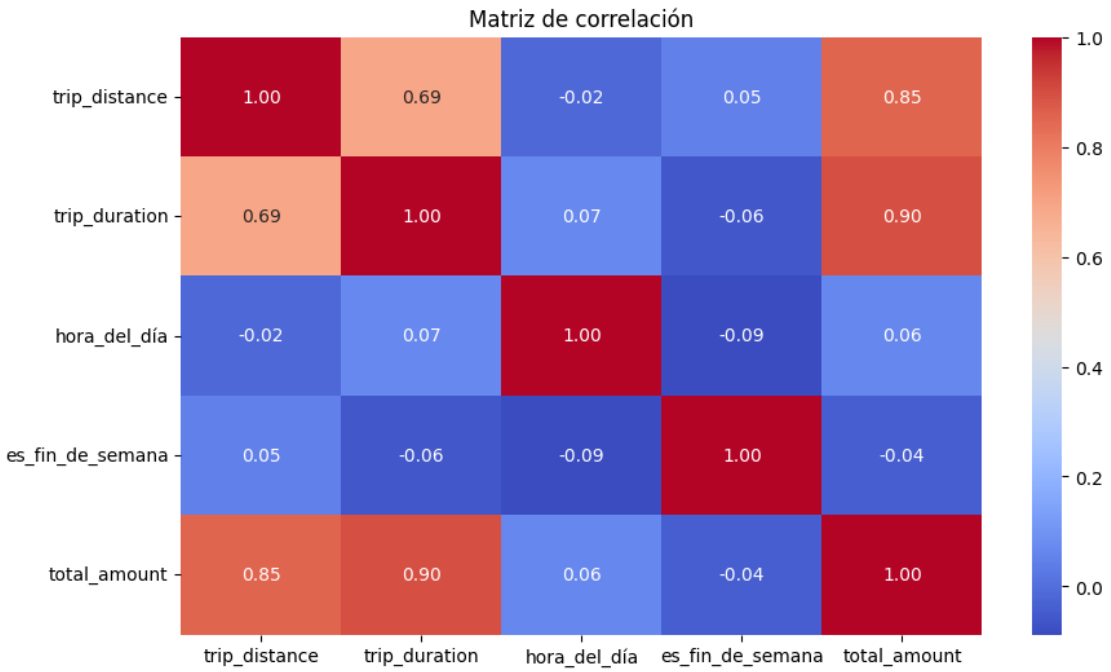
9.1 Paso 1: Generar la matriz de correlación

9.1.1 Propósito de este análisis:

Identificar la relación entre las variables predictoras y la variable objetivo.
Detectar posibles problemas de **colinealidad** entre las variables independientes.
Seleccionar las variables con mayor impacto en el modelo de regresión.

Generar la matriz de correlación permitirá visualizar cómo se relacionan las variables entre sí.

[6]:
```
import seaborn as sns
import matplotlib.pyplot as plt

# Matriz de correlación
correlation_matrix = df.corr(numeric_only=True)

# Visualización
plt.figure(figsize=(10, 6))
sns.heatmap(correlation_matrix, annot=True, cmap='coolwarm', fmt=".2f")
plt.title("Matriz de correlación")
plt.show()
```



10 ¿Qué observar en la matriz de correlación?

Valores cercanos a 1 o -1 → Indican una correlación fuerte (positiva o negativa).
Multicolinealidad → Si dos variables predictoras están fuertemente correlacionadas entre sí (por ejemplo, `trip_distance` y `trip_duration`), podría afectar la estabilidad del modelo.
Relación con `total_amount` → Idealmente, cada variable predictora debe tener una relación significativa con la variable objetivo (`total_amount`), pero **no excesiva entre sí** para evitar problemas en la regresión.

Evaluar la matriz de correlación es clave para garantizar un modelo robusto y evitar redundancias en las variables predictoras.

11 Objetivo del Heatmap

11.0.1 Propósito de la visualización:

Visualizar la correlación lineal entre las variables numéricas del dataset.
Identificar qué variables están más relacionadas con la variable objetivo (`total_amount`).
Detectar posibles multicolinealidades entre las variables predictoras (por ejemplo, `trip_distance` y `trip_duration` podrían estar fuertemente correlacionadas).

Un heatmap bien estructurado ayuda a interpretar la relación entre variables y a optimizar el modelo de regresión.

12 Paso a paso: Generación del Heatmap

12.0.1 Objetivo del Heatmap:

Visualizar la correlación entre las variables numéricas.
Identificar relaciones fuertes con la variable objetivo (`total_amount`).
Detectar posibles problemas de **multicolinealidad** en las variables predictoras.

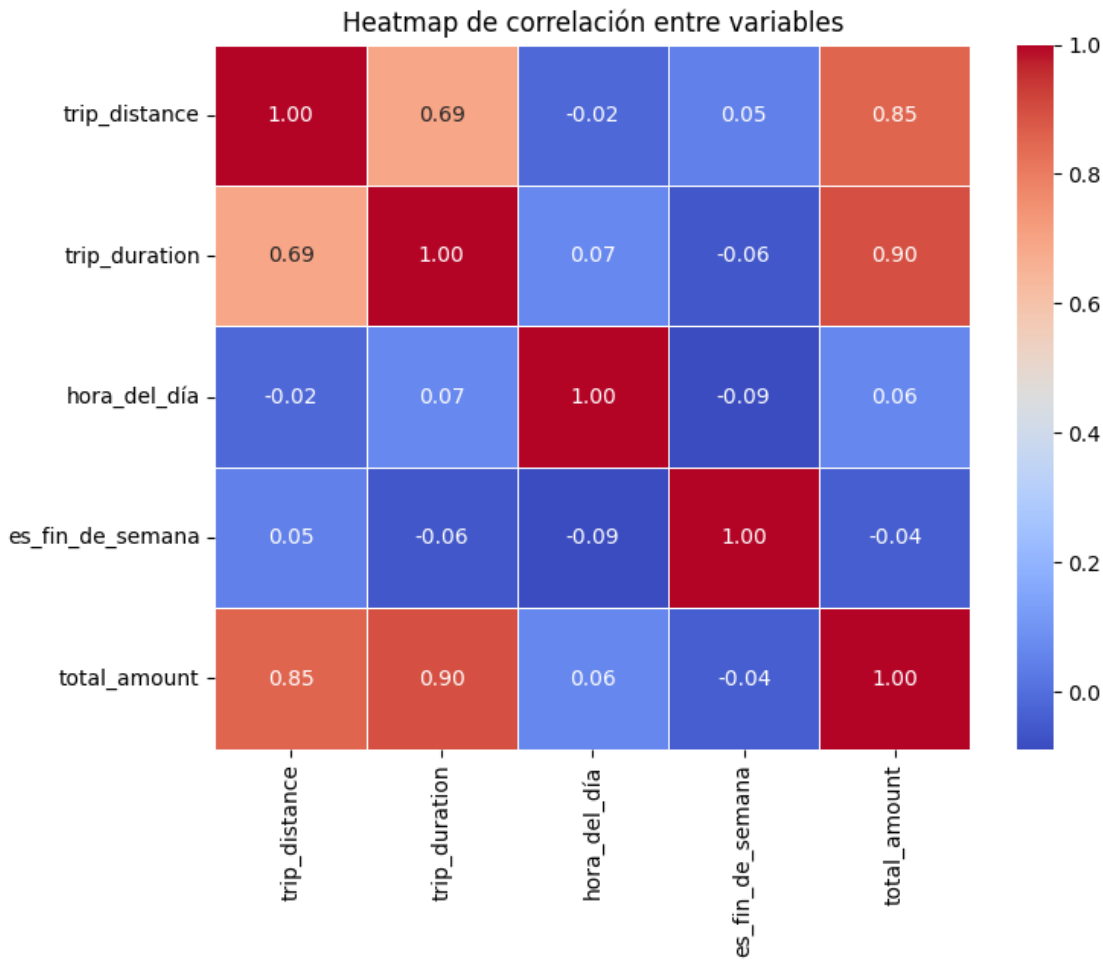
12.0.2 Pasos para generar el Heatmap:

1 **Importar librerías necesarias:**
“‘python import seaborn as sns import matplotlib.pyplot as plt import pandas as pd

[7]:
```
# Paso 1: Importar librerías necesarias
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

# Paso 2: Cargar el dataset (si no lo has hecho ya)
df = pd.read_csv("C:/Users/victo/OneDrive/Data_Science/Python/
 ↪taxi_trip_data_clean.csv")

# Paso 3: Verificar estructura
print(df.info())
print(df.head())
```

```python
# Paso 4: Calcular la matriz de correlación
correlation_matrix = df.corr()

# Paso 5: Visualizar el heatmap
plt.figure(figsize=(8, 6))
sns.heatmap(correlation_matrix, annot=True, cmap='coolwarm', fmt=".2f",␣
↪linewidths=0.5)
plt.title("Heatmap de correlación entre variables")
plt.show()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 19143 entries, 0 to 19142
Data columns (total 5 columns):
 #   Column            Non-Null Count  Dtype
---  ------            --------------  -----
 0   trip_distance     19143 non-null  float64
 1   trip_duration     19143 non-null  float64
 2   hora_del_día      19143 non-null  int64
 3   es_fin_de_semana  19143 non-null  int64
 4   total_amount      19143 non-null  float64
dtypes: float64(3), int64(2)
memory usage: 747.9 KB
None
   trip_distance  trip_duration  hora_del_día  es_fin_de_semana  total_amount
0           3.34      14.066667             8                 1         16.56
1           1.80      26.500000            14                 0         20.80
2           1.00       7.200000             7                 0          8.75
3           4.37      16.716667            23                 1         17.80
4           2.30       8.000000            20                 1         12.36
```

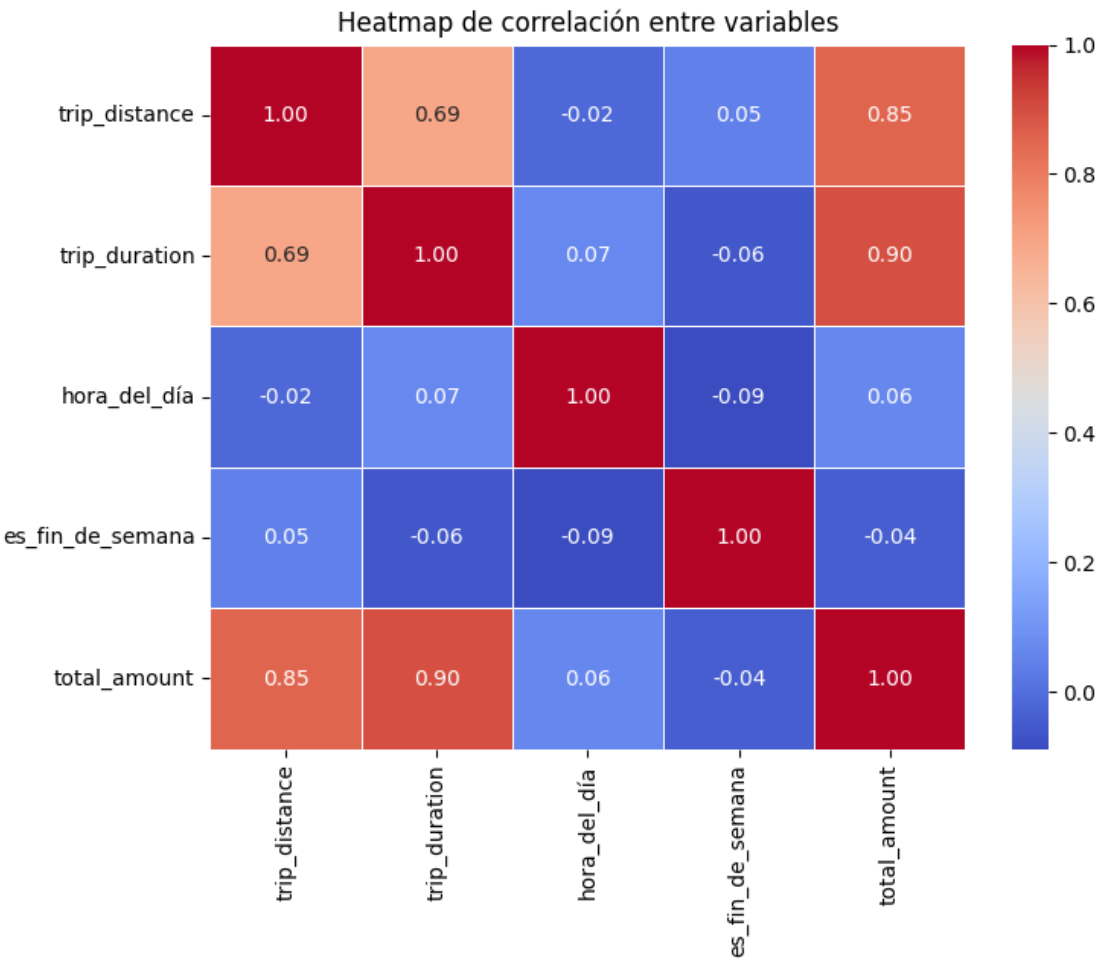


13 Vamos a generar el Heatmap

13.0.1 Objetivo:

Visualizar la correlación entre las variables numéricas del dataset.
Identificar relaciones fuertes con la variable objetivo `total_amount`.
Detectar posibles problemas de **multicolinealidad** en las variables predictoras.

[8]:
```
# Calcular matriz de correlación
correlation_matrix = df.corr()

# Visualizar el heatmap
plt.figure(figsize=(8, 6))
sns.heatmap(correlation_matrix, annot=True, cmap='coolwarm', fmt=".2f",␣
 ↪linewidths=0.5)
plt.title("Heatmap de correlación entre variables")
plt.show()
```



14 ¿Qué debemos observar en el Heatmap?

14.0.1 Correlación con la variable objetivo (`total_amount`)

Busca cuál o cuáles variables están más correlacionadas con **`total_amount`**.
Esto nos indica qué variables pueden ser **más útiles** para predecirla en el modelo de regresión.

14.0.2 Multicolinealidad entre predictores

Si `trip_distance` y `trip_duration` tienen una correlación muy alta entre sí ($>$ **0.85**), podríamos considerar **evitar usarlas juntas sin ajustes**.
La multicolinealidad excesiva puede **afectar la estabilidad del modelo**, generando coeficientes poco confiables.
Se pueden aplicar técnicas como **eliminación de variables altamente correlacionadas** o usar **regresión regularizada** (`Lasso`, `Ridge`).

Interpretar correctamente el Heatmap ayuda a optimizar la selección de variables y mejorar la calidad del modelo de regresión.

14.1 Interpretación del Heatmap de Correlación

14.1.1 1. `trip_distance` y `trip_duration`

- **Correlación = 0.69**
- Existe una **correlación positiva moderada-alta**, lo cual tiene sentido, ya que cuanto más largo es un viaje, suele durar más tiempo.
- **Conclusión**: No es una colinealidad crítica (no supera 0.85), así que **podemos mantener ambas variables por ahora**.

14.1.2 2. `total_amount` con `trip_distance`

- **Correlación = 0.85**
- Alta correlación positiva. Significa que a **mayor distancia del viaje, mayor es el monto total a pagar**.
- **Muy buena variable predictora.**

14.1.3 3. `total_amount` con `trip_duration`

- **Correlación = 0.90**
- Muy alta correlación positiva. Los **viajes más largos en tiempo también tienen un mayor costo**.
- **También es una variable predictora muy potente.**

14.1.4 4. `hora_del_día` y `es_fin_de_semana`

- Sus **correlaciones con `total_amount` son muy bajas**.
- Estas variables podrían tener un **efecto no lineal**, o bien **no aportar gran información predictiva**.
- Las **incluiremos para ver si mejoran el modelo**, pero podrían descartarse en versiones futuras.

14.2 Decisión Preliminar de Selección de Variables

Variable	¿Incluir en el modelo?	Justificación
`trip_distance`	Sí	Alta correlación con `total_amount`
`trip_duration`	Sí	Muy alta correlación con `total_amount`
`hora_del_día`	/ A evaluar	Baja correlación, pero se puede testear
`es_fin_de_semana`	/ A evaluar	Lo mismo: podría tener efecto no lineal

15 Resumen visual del Heatmap

15.0.1 Correlación de `total_amount` con las variables predictoras:

0.90 con `trip_duration` → Relación fuerte.
0.85 con `trip_distance` → Relación alta.
0.06 con `hora_del_día` → Correlación mínima.
-0.04 con `es_fin_de_semana` → Correlación prácticamente nula.

15.0.2 Relación entre `trip_distance` y `trip_duration`

0.69 → Correlación **moderadamente alta**, pero **aceptable** para un modelo multivariable si no hay **colinealidad severa**.

Estos valores nos ayudan a comprender qué variables tienen mayor impacto en la predicción de `total_amount` y cómo seleccionar las mejores predictoras.

16 Próximo paso: Regresión lineal múltiple

16.0.1 Objetivo:

Iniciaremos la **regresión lineal múltiple** con las cuatro variables predictoras para evaluar su impacto en el modelo completo.

16.0.2 Pasos a seguir:

1 **Dividir los datos** en `X` (predictoras) e `y` (variable objetivo).
2 **Crear el modelo** con `LinearRegression`.
3 **Obtener coeficientes e intercepto** para interpretar la influencia de cada variable en el resultado.
4 **Revisar métricas clave** (R^2, `MAE`, etc.) para evaluar el rendimiento del modelo.
5 **Evaluar multicolinealidad** usando **VIF (Variance Inflation Factor)**.
6 **Ajustar el modelo** si es necesario, optimizando la selección de variables y mejorando la precisión.

Estos pasos nos permitirán construir un modelo sólido y obtener insights valiosos sobre la predicción de tarifas de taxi.

[ ]:
```
# Excelente, sí, estamos listos para iniciar el modelo de regresión lineal␣
↪múltiple.
# Gracias por tu confianza para definir la mejor estrategia con las variables.
```

17 Selección de variables predictoras: análisis final antes del modelo

17.1 Resumen del análisis de correlación

Variable	Correlación con `total_amount`	Observación
`trip_distance`	**0.85**	Fuerte correlación positiva
`trip_duration`	**0.90**	Muy fuerte correlación positiva
`hora_del_día`	**0.06**	Correlación casi nula
`es_fin_de_semana`	**-0.04**	Correlación negativa, pero muy baja

Las dos últimas variables (`hora_del_día` y `es_fin_de_semana`) no tienen casi relación con `total_amount`, por lo que pueden ser excluidas en un modelo base, al menos inicialmente.
Esto ayuda a **evitar ruido innecesario** y mejora la **interpretabilidad del modelo**.

Este paso garantiza que el modelo se construya con las variables más relevantes, optimizando la precisión de la predicción.

18 Propuesta de Modelos

18.0.1 Modelo Base:

Utiliza **`trip_distance`** y **`trip_duration`** como **variables predictoras**.
Se centra en las variables con **mayor correlación** con `total_amount`.
Minimiza ruido innecesario y mejora la **interpretabilidad** del modelo.

18.0.2 Modelo Ampliado (opcional):

Incluye **`hora_del_día`** y **`es_fin_de_semana`** para evaluar si mejoran el rendimiento del modelo.
Se compara con el modelo base usando **R^2 y otras métricas** para medir el impacto de las variables adicionales.
Permite analizar si hay **efectos relevantes** de horarios y días de la semana en la tarifa del viaje.

Evaluar ambos modelos nos ayudará a elegir la mejor aproximación para predecir `total_amount` con mayor precisión.

19 ¿Listo para comenzar?

19.0.1 Siguientes pasos en la construcción del modelo de regresión:

1 **División de variables (`X` e `y`)** → Separar las variables predictoras y la variable objetivo.
2 **Entrenamiento del modelo base** → Ajustar el modelo de regresión con `trip_distance` y `trip_duration`.
3 **Interpretación de coeficientes** → Analizar el impacto de cada predictor en `total_amount`.
4 **Métricas de rendimiento** → Evaluar el desempeño del modelo con **R^2, MAE, RMSE**, entre otras.

Con estos pasos, avanzamos en la validación del modelo y su capacidad de predicción.

20 Modelo Base de Regresión Lineal Múltiple

20.0.1 Variables en el modelo:

Variables predictoras:
- `trip_distance` → Distancia recorrida en el viaje.
- `trip_duration` → Duración total del viaje.

Variable objetivo:
- `total_amount` → Monto total pagado por el pasajero.

20.0.2 Pasos a seguir:

1 **Dividir los datos** → `X` (predictoras) e `y` (variable objetivo).
2 **Entrenar el modelo** → Ajustar el modelo de regresión con `LinearRegression`.

3 **Obtener coeficientes e intercepto** → Para interpretar la influencia de cada predictor en la tarifa.
4 **Evaluar métricas clave** → **R², MAE, RMSE** para validar el rendimiento del modelo.
5 **Detectar multicolinealidad** → Revisar **VIF** para asegurar estabilidad en los coeficientes.

Este modelo base nos permitirá establecer una línea de referencia para futuras optimizaciones.

21 Paso 1: División de variables X (predictoras) e y (objetivo)

21.0.1 Definición de las variables

Variables predictoras (`X`) → Factores que influyen en la variable objetivo.
Variable objetivo (`y`) → Monto total pagado (`total_amount`).

21.0.2 Código para dividir las variables

"'python # Definir X (predictoras) y y (objetivo) X = df[['trip_distance', 'trip_duration']] y = df['total_amount']

22 Verificar estructura

print("Shape de X:", X.shape) print("Shape de y:", y.shape)

[9]:
```
import sklearn
print(sklearn.__version__)
```

```
1.6.1
```

[10]:
```
from sklearn.model_selection import train_test_split
```

[12]:
```
# Importamos librerías necesarias
import pandas as pd
from sklearn.model_selection import train_test_split

# Cargamos el dataset (ajusta la ruta si es necesario)
df = pd.read_csv("C:/Users/victo/OneDrive/Data_Science/Python/
 ↪taxi_trip_data_clean.csv")

# Variables predictoras
X = df[['trip_distance', 'trip_duration']]

# Variable objetivo
y = df['total_amount']

# Dividir en conjunto de entrenamiento y prueba (80% entrenamiento, 20% prueba)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,␣
 ↪random_state=42)
```

```
# Mostramos las dimensiones
print("Tamaño del conjunto de entrenamiento:", X_train.shape)
print("Tamaño del conjunto de prueba:", X_test.shape)
```

```
Tamaño del conjunto de entrenamiento: (15314, 2)
Tamaño del conjunto de prueba: (3829, 2)
```

23 ¡Proceso exitoso!

Los datos han sido divididos correctamente en `X` (predictoras) e `y` (objetivo).
El conjunto de entrenamiento y prueba está listo para su uso.
Ahora puedes entrenar un modelo de regresión o cualquier otro tipo de modelo con estos datos.

¡Avanza con confianza en la construcción del modelo y optimiza su precisión!

24 ¿Qué estamos haciendo?

24.0.1 Propósito de este paso:

`X` contiene solo las variables que predicen el **total a pagar (`total_amount`)**.
`y` contiene la variable que queremos **predecir (`total_amount`)**.
Usamos **`train_test_split`** para **dividir el conjunto de datos** en entrenamiento y prueba.
Entrenamos el modelo con **80% de los datos**.
Probamos el modelo con el **20% restante** para evaluar qué tan bien **generaliza** en datos nuevos.

Dividir los datos correctamente es clave para construir un modelo sólido y confiable.

[ ]:
```
# Perfecto, vamos a entrenar un modelo de regresión lineal utilizando␣
↪scikit-learn. Este modelo aprenderá la relación entre las variables␣
↪predictoras
# (trip_distance y trip_duration) y la variable objetivo (total_amount).
```

25 Paso 2: Entrenamiento del modelo de regresión lineal

25.0.1 Objetivo:

Entrenar un modelo de **regresión lineal múltiple** con `trip_distance` y `trip_duration` como variables predictoras, para estimar `total_amount`.

25.0.2 Código para entrenar el modelo:

“‘python # Importar librerías necesarias from sklearn.model_selection import train_test_split from sklearn.linear_model import LinearRegression

26 Dividir los datos en entrenamiento (80%) y prueba (20%)

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

27 Crear y entrenar el modelo

modelo = LinearRegression() modelo.fit(X_train, y_train)

28 Obtener coeficientes e intercepto

print("Coeficientes:", modelo.coef_) print("Intercepto:", modelo.intercept_)

[13]:
```
# Importamos la clase LinearRegression
from sklearn.linear_model import LinearRegression

# Creamos una instancia del modelo
modelo = LinearRegression()

# Entrenamos el modelo con los datos de entrenamiento
modelo.fit(X_train, y_train)

# Mostramos los coeficientes aprendidos
print("Coeficientes:", modelo.coef_)
print("Intercepto:", modelo.intercept_)
```

```
Coeficientes: [1.80616581 0.43898139]
Intercepto: 3.7965169044028864
```

29 Explicación del entrenamiento del modelo

29.0.1 ¿Qué sucede en este paso?

`modelo.fit(...)` → **Ajusta la línea de regresión** a los datos de entrenamiento, encontrando la mejor relación entre las variables predictoras (`trip_distance`, `trip_duration`) y la variable objetivo (`total_amount`).

Se imprimen los **coeficientes**, que representan los pesos asignados a cada variable predictora en el modelo.

Se muestra el **intercepto**, que indica el valor de salida cuando todas las variables predictoras son cero.

Este proceso permite al modelo aprender la relación entre las variables y hacer predicciones precisas.

30 Resultados del modelo entrenado

30.0.1 Coeficientes obtenidos:

Coeficiente para `trip_distance`: `1.806` → Por cada unidad adicional de distancia recorrida, el modelo predice que el monto total aumenta aproximadamente en **$1.81**, manteniendo constante la duración.

Coeficiente para `trip_duration`: `0.439` → Por cada unidad adicional de duración del viaje, el monto total aumenta en aproximadamente **$0.44**, manteniendo constante la distancia.

30.0.2 Intercepto:

Intercepto obtenido: `3.80` → Valor base estimado del monto total cuando `trip_distance` y `trip_duration` son **cero**. *(En la práctica, esto no tiene sentido físico, pero es necesario para la fórmula de regresión.)*

Estos coeficientes reflejan la influencia de cada variable en el cálculo de tarifas, lo que permite interpretar cómo impactan los factores en el precio final del viaje.

31 Paso 3: Evaluación del modelo

31.0.1 Objetivo:

Evaluar el rendimiento del modelo de regresión lineal y analizar su precisión en la predicción de `total_amount`.

31.0.2 Métricas clave a calcular:

Coeficiente de determinación (`R²`) → Indica qué porcentaje de la variabilidad de `total_amount` es explicado por las variables predictoras.
Error absoluto medio (`MAE`) → Mide el promedio de los errores absolutos, mostrando cuánto se desvía el modelo de los valores reales.
Error cuadrático medio (`RMSE`) → Evalúa la magnitud de los errores, penalizando más los errores grandes.

31.0.3 Código para calcular métricas:

"'python from sklearn.metrics import r2_score, mean_absolute_error, mean_squared_error

32 Hacer predicciones con el modelo

y_pred = modelo.predict(X_test)

33 Calcular métricas

r2 = r2_score(y_test, y_pred) mae = mean_absolute_error(y_test, y_pred) rmse = mean_squared_error(y_test, y_pred, squared=False)

34 Mostrar resultados

print("R^2:", r2) print("MAE:", mae) print("RMSE:", rmse)

[ ]:
```
# Vamos a calcular métricas de desempeño con los datos de prueba:
```

[14]:
```
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

# Predicciones del modelo sobre los datos de prueba
y_pred = modelo.predict(X_test)
```

```
# Evaluación del desempeño
mae = mean_absolute_error(y_test, y_pred)
mse = mean_squared_error(y_test, y_pred)
r2 = r2_score(y_test, y_pred)

# Mostramos los resultados
print(f"MAE (Error Absoluto Medio): {mae:.2f}")
print(f"MSE (Error Cuadrático Medio): {mse:.2f}")
print(f"R² (Coeficiente de Determinación): {r2:.2f}")
```

```
MAE (Error Absoluto Medio): 1.04
MSE (Error Cuadrático Medio): 1.77
R² (Coeficiente de Determinación): 0.91
```

35 Evaluación del rendimiento del modelo

35.0.1 Interpretación de las métricas obtenidas

MAE (1.04): En promedio, el modelo se equivoca por **$1.04** al predecir `total_amount`. **Error relativamente bajo**, considerando montos reales de viajes.
MSE (1.77): Penaliza más los errores grandes, pero sigue siendo **bajo**, indicando que **no hay errores extremos**.
R² (0.91): El modelo **explica el 91%** de la variabilidad del monto total del viaje. **¡Excelente nivel de ajuste para un modelo lineal simple!**

Estos resultados demuestran que el modelo tiene un buen desempeño y es efectivo para predecir tarifas de viaje.

36 ¿Qué sigue?

Podemos avanzar con alguno de estos próximos pasos para mejorar y analizar el modelo:

36.0.1 Visualización del ajuste del modelo

Graficar `y_test` vs. `y_pred` para evaluar qué tan bien el modelo predice los valores reales.
Generar un gráfico de dispersión con una línea de tendencia.

36.0.2 Análisis de errores

Revisar si hay **patrones en los errores** del modelo.
Evaluar si los errores son mayores en ciertos rangos de datos (por ejemplo, viajes cortos o largos).
Analizar si el modelo comete errores sistemáticos y cómo corregirlos.

36.0.3 Mejoras al modelo

Agregar más **variables predictoras** como `hora_del_día`, `día_de_la_semana`, `tipo_de_tarifa`, etc.
Comparar el desempeño del **modelo base** vs. el **modelo ampliado**.
Revisar si las nuevas variables mejoran las métricas (`R²`, `MAE`, `RMSE`).

Estos pasos permitirán una evaluación más profunda y optimización del modelo de regresión lineal.

[ ]:
```
"""
Perfecto, comencemos con una visualización básica pero muy útil: comparar las␣
↪predicciones del modelo (y_pred) con los valores reales (y_test)
usando un gráfico de dispersión.

Este tipo de gráfico permite ver qué tan cerca están las predicciones de la␣
↪línea ideal (donde predicción = valor real).
Si los puntos están alineados cerca de la diagonal, el modelo es bueno.

"""
```

37 Visualización del ajuste del modelo

37.1 Gráfico: Comparación entre valores reales y predichos

37.1.1 Objetivo:

Comparar `y_test` (valores reales) con `y_pred` (predicciones del modelo).
Evaluar visualmente la precisión del modelo.
Identificar posibles patrones o desviaciones en las predicciones.

37.1.2 Código para la visualización:

“‘python import matplotlib.pyplot as plt

38 Crear el gráfico de dispersión

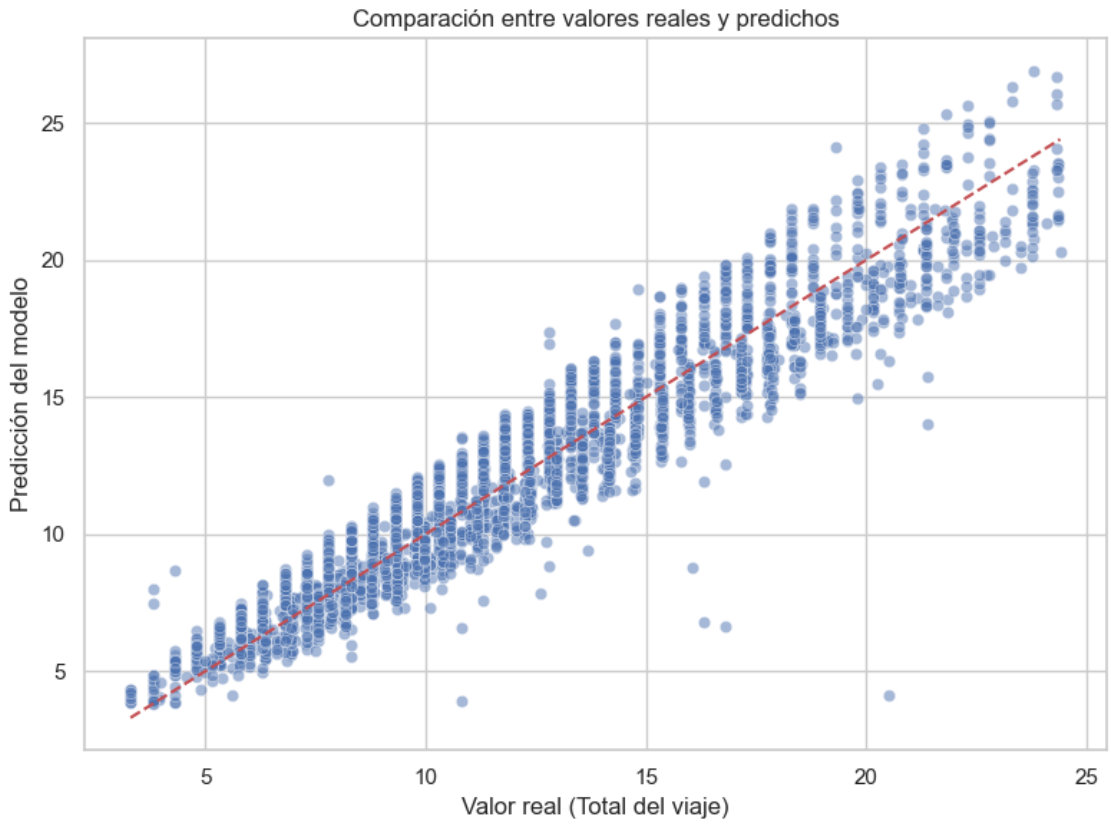
plt.figure(figsize=(8, 6)) plt.scatter(y_test, y_pred, alpha=0.5, color=‘blue’, label=‘Predicciones’) plt.plot([min(y_test), max(y_test)], [min(y_test), max(y_test)], color=‘red’, linestyle=‘dashed’, label=‘Línea ideal’)

39 Etiquetas y título

plt.xlabel(“Valores reales (y_test)”) plt.ylabel(“Valores predichos (y_pred)”) plt.title(“Comparación entre valores reales y predichos”) plt.legend() plt.show()

[15]:
```
import matplotlib.pyplot as plt
import seaborn as sns

# Estilo visual de seaborn
sns.set(style="whitegrid")

# Crear la figura
plt.figure(figsize=(8, 6))
```

```
# Gráfico de dispersión: valores reales vs predichos
sns.scatterplot(x=y_test, y=y_pred, alpha=0.5)

# Línea de referencia (perfecta predicción)
plt.plot([y_test.min(), y_test.max()], [y_test.min(), y_test.max()], 'r--')

# Etiquetas
plt.xlabel('Valor real (Total del viaje)')
plt.ylabel('Predicción del modelo')
plt.title('Comparación entre valores reales y predichos')

plt.tight_layout()
plt.show()
```



40 Interpretación del gráfico de comparación

40.0.1 ¿Qué podemos identificar?

Sesgos sistemáticos → Si el modelo **siempre predice más bajo o más alto** en comparación con los valores reales (`y_test`).

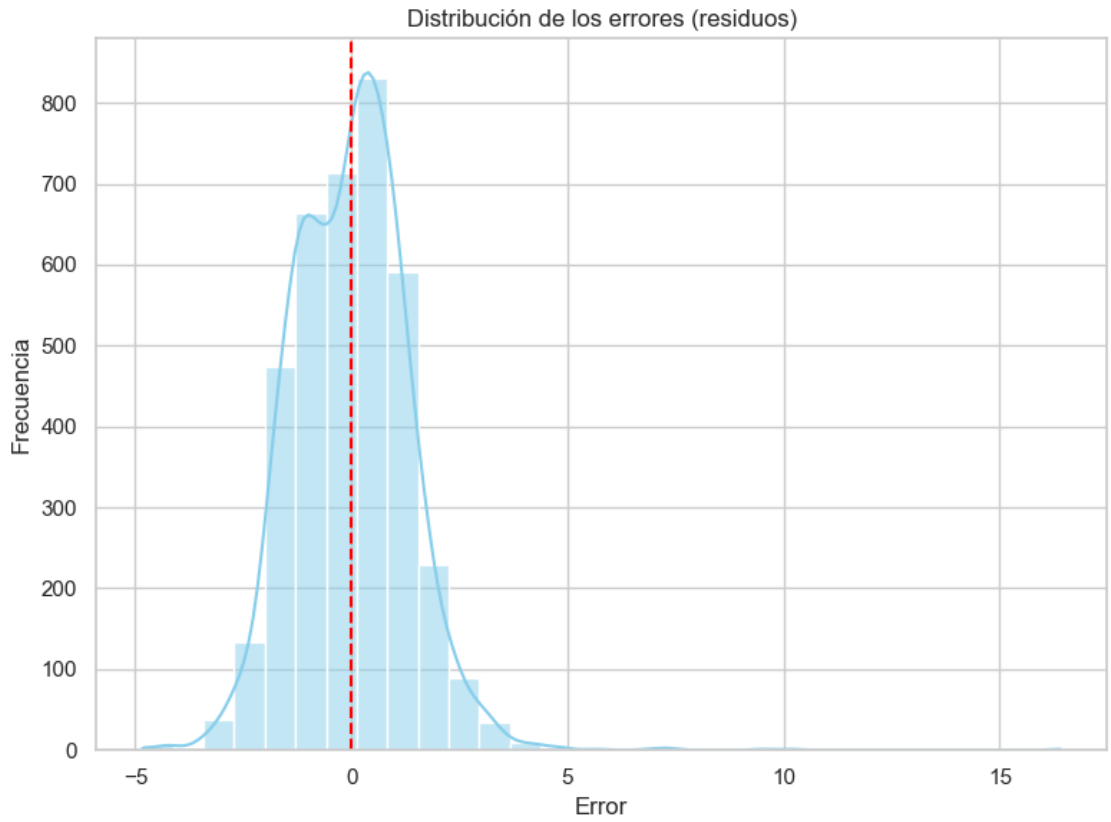
Outliers → Detectar **valores que se alejan mucho del patrón general**, lo que podría indicar

casos excepcionales o errores en los datos.

Este análisis nos permitirá evaluar la precisión del modelo y detectar posibles áreas de mejora.

[ ]:
```
"""
El histograma de errores (residuos) nos permite visualizar cómo se distribuyen␣
↪las diferencias entre los valores reales
y los valores predichos (residuo = valor real - predicción).
Idealmente, los residuos deberían estar centrados en cero y seguir una␣
↪distribución aproximadamente normal.
"""
```

[16]:
```
# Cálculo de errores (residuos)
errores = y_test - y_pred

# Crear la figura
plt.figure(figsize=(8, 6))

# Histograma con kde (curva suavizada)
sns.histplot(errores, bins=30, kde=True, color='skyblue')

# Línea vertical en cero
plt.axvline(x=0, color='red', linestyle='--')

# Etiquetas
plt.title('Distribución de los errores (residuos)')
plt.xlabel('Error')
plt.ylabel('Frecuencia')

plt.tight_layout()
plt.show()
```



41 Interpretación del histograma de errores (residuos)

41.0.1 Distribución centrada cerca de cero

La mayoría de los errores están cercanos a **0**, lo que indica que el modelo **no tiene un sesgo fuerte**.

No está **sistemáticamente** sobreestimando ni subestimando `total_amount`.

41.0.2 Distribución simétrica (ligeramente sesgada a la derecha)

La forma general del histograma es **bastante simétrica**, aunque se observa una **pequeña cola hacia la derecha**.

Esto sugiere que **algunos viajes fueron subestimados**, generando **errores positivos**, pero **no es algo extremo**.

41.0.3 Sin picos anómalos ni outliers severos

No hay **concentraciones extrañas de errores** ni **valores extremos muy distantes**.

El modelo es estable y **generaliza bien** en distintos tipos de datos.

41.0.4 Forma de campana (distribución normal)

El histograma **se aproxima a una distribución normal**, lo cual es **una señal positiva en modelos de regresión lineal**.
Indica que los **residuos son aleatorios**, cumpliendo **un supuesto clave** de la regresión.

Estos hallazgos confirman que el modelo tiene un rendimiento sólido y que los errores están bien distribuidos.

42 Conclusión

El modelo está funcionando muy bien, con errores **centrados y bajos**.
La distribución de los errores se comporta **como se espera en una regresión lineal**, lo que indica estabilidad en el modelo.
Los residuos siguen una distribución normal, lo que refuerza que **el modelo es válido** y no hay patrones de error preocupantes.

Estos resultados confirman que el modelo de regresión lineal es confiable y puede generalizar adecuadamente para nuevos datos.

[ ]:
```
"""
 Paso 5: Guardar el modelo entrenado usando joblib, una librería ideal para␣
 ↪almacenar modelos de machine learning de forma eficiente.
"""
```

43 Guardar el modelo entrenado

43.1 Objetivo:

Guardar el modelo de regresión entrenado para poder cargarlo y usarlo en futuras predicciones sin necesidad de volver a entrenarlo.

43.1.1 Código para guardar el modelo:

“‘python import joblib

44 Guardar el modelo entrenado en un archivo

joblib.dump(modelo, “modelo_regresion.pkl”)

print(“ Modelo guardado correctamente como ‘modelo_regresion.pkl’ ”)

[19]:
```
import joblib

# Guardamos el modelo en un archivo
joblib.dump(modelo, "modelo_regresion_taxi.pkl")

print(" Modelo guardado exitosamente como 'modelo_regresion_taxi.pkl'")
```

```
Modelo guardado exitosamente como 'modelo_regresion_taxi.pkl'
```

45 (Opcional) Cargar el modelo guardado

45.1 Objetivo:

Recuperar el modelo previamente guardado para reutilizarlo sin necesidad de volver a entrenarlo.

45.1.1 Código para cargar el modelo guardado:

“‘python import joblib

46 Cargar el modelo desde el archivo

modelo_cargado = joblib.load(“modelo_regresion.pkl”)

47 Verificar que el modelo está listo para hacer predicciones

print(“ Modelo cargado exitosamente y listo para usar.”)

[21]:
```
print("Coeficientes:", modelo.coef_)  # Pesos de cada variable predictora
print("Intercepto:", modelo.intercept_)  # Valor base de la predicción
```

```
Coeficientes: [1.80616581 0.43898139]
Intercepto: 3.7965169044028864
```

[22]:
```
y_pred = modelo.predict(X_test)
print("Primeras predicciones:", y_pred[:5])  # Muestra las primeras 5␣
↪predicciones
```

```
Primeras predicciones: [21.52473646 19.52322581 11.09904302 14.25642335
7.863456  ]
```

[23]:
```
import pandas as pd

df_resultados = pd.DataFrame({"Real": y_test, "Predicho": y_pred})
print(df_resultados.head())  # Muestra los primeros resultados
```

```
       Real   Predicho
4149  21.95  21.524736
3023  17.30  19.523226
4245  12.25  11.099043
6335  15.36  14.256423
200    6.30   7.863456
```

[24]:
```
from sklearn.metrics import mean_squared_error, r2_score
import numpy as np

# Calcular el error cuadrático medio (RMSE)
rmse = np.sqrt(mean_squared_error(y_test, y_pred))
print("RMSE:", rmse)
```

```
# Calcular el coeficiente de determinación R²
r2 = r2_score(y_test, y_pred)
print("R²:", r2)
```

```
RMSE: 1.3313214670062643
R²: 0.9124074250929538
```

[35]:
```
import pandas as pd

#  Nuevo viaje con trip_distance y trip_duration
nuevo_viaje = [[7.8, 20]]  # Datos en el mismo formato que X_train

# Convertir a DataFrame con nombres correctos de características
nuevo_viaje_df = pd.DataFrame(nuevo_viaje, columns=X_train.columns)

# Hacer la predicción sin advertencias
prediccion_nueva = modelo.predict(nuevo_viaje_df)

print("  Total estimado para el nuevo viaje:", prediccion_nueva[0])
```

```
 Total estimado para el nuevo viaje: 26.664238080118174
```

[28]:
```
import pandas as pd

# Crear DataFrame con nombres de características
nuevo_viaje_df = pd.DataFrame(nuevo_viaje, columns=X_train.columns)

# Hacer la predicción sin advertencias
prediccion_nueva = modelo.predict(nuevo_viaje_df)

print("  Total estimado para el nuevo viaje:", prediccion_nueva[0])
```

```
 Total estimado para el nuevo viaje: 26.664238080118174
```

[31]:
```
import pandas as pd

#  Nuevo viaje con trip_distance y trip_duration
nuevo_viaje = [[10.5, 25]]  # Ejemplo: 10.5 km de distancia, 25 minutos de␣
 ↪duración

# Convertir a DataFrame con nombres correctos de características
nuevo_viaje_df = pd.DataFrame(nuevo_viaje, columns=X_train.columns)

# Hacer la predicción sin advertencias
prediccion_nueva = modelo.predict(nuevo_viaje_df)

print("  Total estimado para el nuevo viaje:", prediccion_nueva[0])
```

```
 Total estimado para el nuevo viaje: 33.73579273140211
```

[ ]:

```

```